

背景

批次单是生产任务最小组织单元。目前已实现批次单管理的基本功能。

 [\[PRD\]产线控制软件V0.1](#)

 [\[FeatureDoc\]生产任务导入B0.1](#)

目标

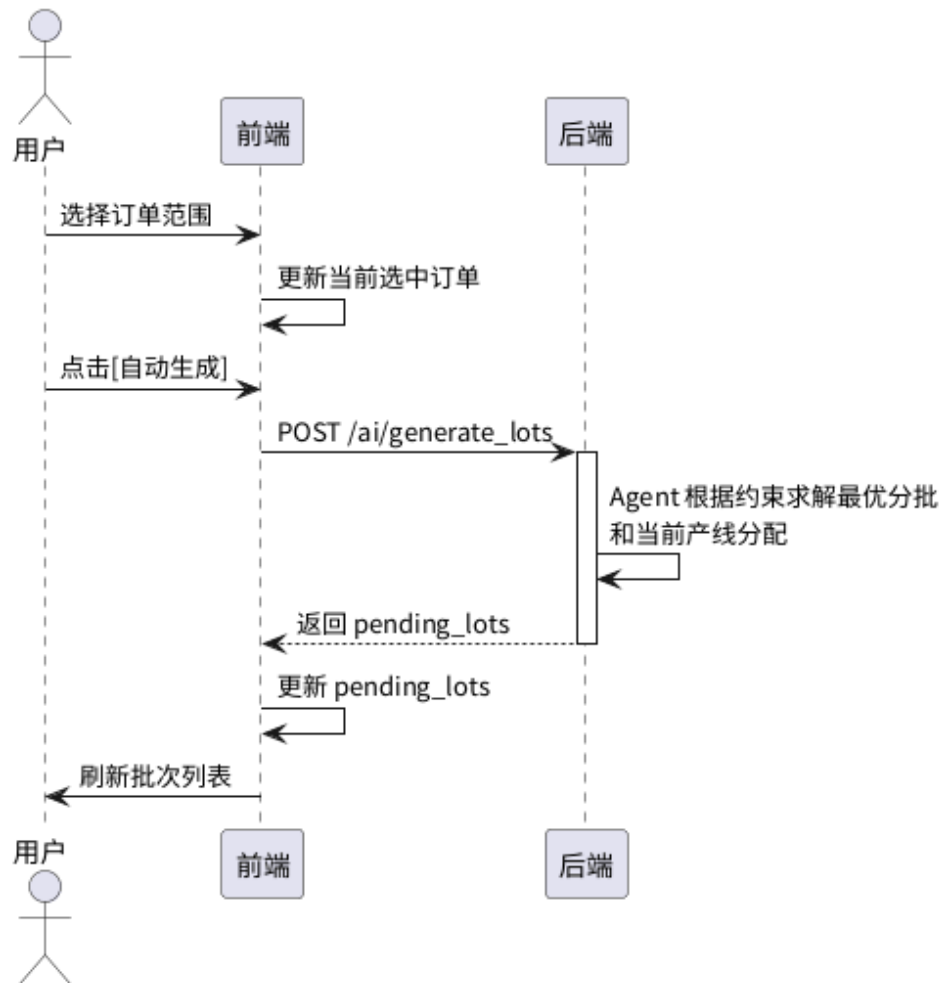
实现批次单管理智能辅助的最小闭环：

1. 批次单的自动生成
2. 批次单的自动校验

方案

1. 功能描述
 - a. 批次单的自动生成
 - i. 界面：生产任务导入界面
 - ii. 流程：

批次单自动生成



1. 用户在生产订单列表选择一个或多个生产订单作为范围。若未选择，则默认全部当前订单。
2. 用户点击 [自动生成] 按钮。前端将当前选中订单和产线约束发送给后端。
3. 后端调用 Agent，按批次约束和产线约束生成候选批次。
4. 后端返回 pending_lots。前端刷新批次列表。

iii. API 设计：

1. POST /ai/generate_lots 自动生成批次单

a. 格式

i. 入参:

```
{
    selected_orders: list[str], 每个元素为 order_id
}
```

ii. 出参：

```
{
  passed: bool , 产生 lot 是否成功
  message: str , 相关信息
  pending_lots: [
    {
      lot_header: {
        lot_id : str , 生产批次号
        source_order_id : list[str] , 关联的生产
        订单号
        production_line_id : str, 产线编号
      }
      lot_line: [
        {
          lot_line_id : str, 当前批次行 id
          color_separation_plan : str , 当前
          sku 分色方案
          source_order_id : str , 关联订单号 ,
          source_order_line_id :str ,关联生产
          订单行号
          color_separation_plan: str 当前 sku
          分色方 案版本 id
          status: str , 批次行状态 created
        }
      ]
    }
  ]
}
```

b. 测试用例：

i. 请求：

```
{
  selected_orders: ["PO-20260206-01"]
}
```

ii. 反馈：

```
{
  passed: "true",
  message: "成功创建 LOT-20260206-001" ,
  pending_lots: [
    {
      lot_header: {
        lot_id: "LOT-20260206-001",
        source_order_ids: ["PO-20260206-01"],
        production_line_id: F01-SP01
      },
      lot_line: [
        {
          lot_id: "LOT-20260206-001",
          lot_line_id: 1,
          source_order_id: "PO-20260206-01",
          source_order_line_id: 1,
          sku: "8Pro",
          color: "神行橘",
          size: 43,
          color_separation_plan: CS-8Pro-神行橘-43
        },
        {
          lot_id: "LOT-20260206-001",
          lot_line_id: 2,
          source_order_id: "PO-20260206-01",
          source_order_line_id: 1,
          sku: "8Pro",
          color: "神行橘",
          size: 43,
          color_separation_plan: CS-8Pro-神行橘-43
        }
      ]
    }
  ],
  quantity_planned: 1200
}
```

```

    }
  ]
}

```

c. Langgraph 节点设计：

- i. 节点 :load_context -> precheck_orders -> build_candidates
-> choose_plan -> validate -> finalize

- ii. 参考流程代码："""

```

graph = StateGraph(AppState)

graph.add_node("load_context", load_context)
graph.add_node("precheck_orders", precheck_orders)
graph.add_node("build_candidates", build_candidates)
graph.add_node("choose_plan", choose_plan)
graph.add_node("validate", validate)
graph.add_node("repair_once", repair_once)
graph.add_node("finalize", finalize)
graph.set_entry_point("load_context")
graph.add_edge("load_context", "precheck_orders")
graph.add_conditional_edges(
    "precheck_orders",
    route_after_precheck,
    { "finalize": "finalize", "build_candidates":
"build_candidates"},
)
graph.add_conditional_edges(
    "build_candidates",
    route_after_build_candidates,
    {"finalize": "finalize", "choose_plan": "choose_plan"},
)
graph.add_edge("choose_plan", "validate")
graph.add_conditional_edges(
    "validate",

```

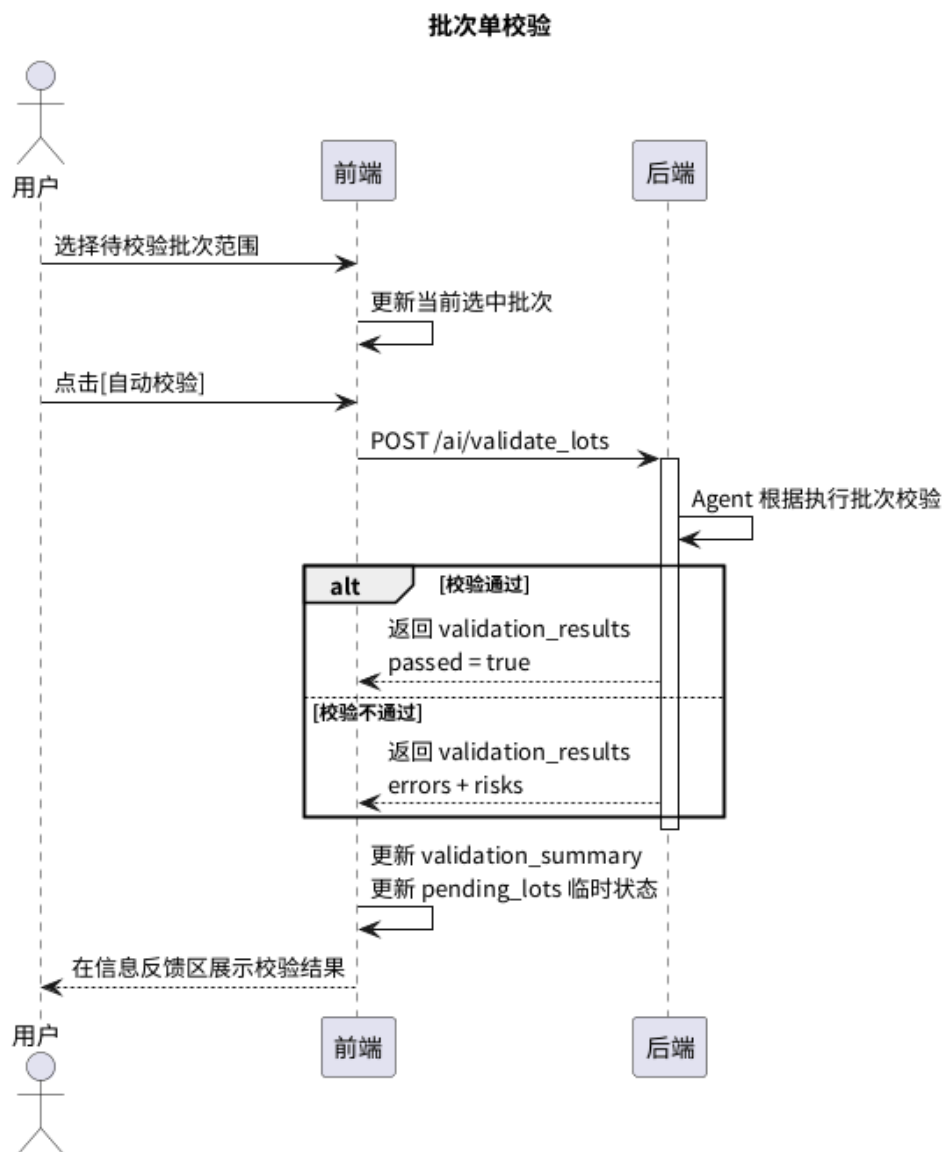
```

route_after_validate,
{"finalize": "finalize", "repair_once": "repair_once"},
)
graph.add_edge("repair_once", "validate")
graph.add_edge("finalize", END)
"""

```

b. 批次单自动校验

- i. 界面：生产任务导入界面
- ii. 关键步骤：



1. 用户在批次列表选择一个或多个批次作为范围。若未选择，则默认全部 pending_lots。

2. 用户点击 [自动校验] 按钮。前端将目标批次和产线约束发送给后端。
3. 后端调用 Agent，执行混线约束校验与产线支持性校验。后端返回校验结果。
4. 前端更新信息反馈区，并把通过的 pending lot 标记为前端临时态 validated。

iii. API 设计：

1. POST /ai/validate_lots 验证上传批次信息是否满足批次约束

a. 格式

i. 入参：

```
{
    pending_lots: list[dict]，需验证的批次信息
}
```

ii. 出参：

```
{
    validation_results: [
        {
            lot_id: str，上传验证的 lot_id,
            passed: bool，是否通过校验
            errors: [str]，错误信息，
            risk_info: [str]，风险信息
        }
    ]
}
```

b. 测试用例

i. 请求：

```
{
    pending_lot: [
        {
            lot_header: {
                lot_id: "LOT-20260206-001",
                source_order_ids: ["PO-20260206-01"],
            }
        }
    ]
}
```

```

        production_line_id: "F01-SP01"
    },
    lot_line: [
        {
            lot_id: "LOT-20260206-001",
            lot_line_id: 1
            source_order_id: "PO-20260206-01"
            source_order_line_id: 1
            sku: "8Pro"
            color: "神行橘"
            size: 43
            color_separation_plan: "CS-8Pro- 神
行橘-43"

            quantity_planned: 1200
        }
    ]
}
]
}
ii. 反馈 :
{
    validation_results:[
        {
            lot_id: "LOT-20260206-001",
            passed: true,
            errors: [],
            risk_info: []
        }
    ]
}

```

2. 批次单约束：

- a. 混线约束：
 - i. sku 一致
 - ii. 颜色一致
 - iii. 质检 (fabirc_material / fabric_thickness / basecoat_thickness / ink_thickness / flatness)要求一致
 - iv. 每个 LOT 最多容纳 2 个生产订单行
- b. 产线约束：
 - i. 分色层数 <= 丝印工作站总数 - 3

附录

1. 时序图代码

编号	PlantUML
批次单生成时序图	<pre> @startuml skinparam defaultFontName "Noto Sans CJK SC" title 批次单自动生成 actor 用户 as User participant 前端 as FE participant 后端 as BE User -> FE: 选择订单范围 FE -> FE: 更新当前选中订单 User -> FE: 点击[自动生成] FE -> BE: POST /ai/generate_lots activate BE BE -> BE: Agent 根据约束求解最优分批\n 和当前产线分配 BE --> FE: 返回 pending_lots deactivate BE </pre>

	FE -> FE: 更新 pending_lots FE -> User: 刷新批次列表 @enduml
批次单自动校验	@startuml skinparam defaultFontName "Noto Sans CJK SC" title 批次单校验 actor 用户 as User participant 前端 as FE participant 后端 as BE User -> FE: 选择待校验批次范围 FE -> FE: 更新当前选中批次 User -> FE: 点击[自动校验] FE -> BE: POST /ai/validate_lots activate BE BE -> BE: Agent 根据执行批次校验 alt 校验通过 BE --> FE: 返回 validation_results\passed = true else 校验不通过 BE --> FE: 返回 validation_results\nerrors + risks end deactivate BE FE -> FE: 更新 validation_summary\n 更新 pending_lots 临时状态 FE --> User: 在信息反馈区展示校验结果 @enduml